



Sliding window & Two pointers



Two pointers

Iterating two monotonic pointers across an array to search for a pair of indices satisfying some condition in linear time.

Ex: [Books \(CF 279 B\)](#)

List of N books, $a_1 a_2 \dots a_n$ minutes to read, find the longest contiguous segment of books that can be read within T minutes.



Subarray sum

We are given an array of n positive integers and a target sum T , and we want to find a subarray whose sum is T or report that there is no such subarray.

Ex: 1 3 2 5 1 1 2 3 target $T = 8$

(ideias de solução deste e de vários problemas a seguir por Two Pointers no Capítulo 8 [deste livro](#)).



2SUM problem

Given an array of N numbers and a target sum T , find two array values such that their sum is T , or report that no such values exist.

Ex: 1 4 5 6 7 9 9 10 target $T = 12$



Hipopotomonstrosesquipedaliofobia (SI 2025)

Maior sequência no string que não tem caractere repetido mais de k vezes

[SI 2025 H](#)



Nearest smaller elements

Finding for each array element the nearest smaller element, i.e., the first smaller element that precedes the element in the array (report if there is no such element)

Ex: 1 3 4 2 5 3 4 2



Sliding window

Constant-size subarray that moves from left to right through the array, calculating some information about the elements inside the window.



Sliding window minimum

Maintaining the sliding window minimum,
which means that we should report the smallest value inside each window

Ex: 2 1 4 5 3 4 1 2 size 4

Seção 8.3 do [livro](#) citado anteriormente e também [aqui](#) (com um passo a passo da ideia).



3SUM

Check if a given set of N real numbers contains three elements that sum to zero.



3SUM - Three different arrays

Instead of searching for the 3 numbers in a single array, search for them in 3 different arrays.



3SUM - Three different arrays

Instead of searching for the 3 numbers in a single array, search for them in 3 different arrays.

Consider

$$A[i] = 10A[i] + 1$$

$$B[i] = 10B[i] + 2$$

$$C[i] = 10C[i] - 3$$



Cycle-Finding

Given a function $f: S \rightarrow S$ (that maps a natural number from a finite set S to another natural number in the same finite set S) and an initial value x_0 , the sequence $x_0, x_1 = f(x_0), x_2 = f(x_1), \dots, x_i = f(x_{i-1}), \dots$ must eventually repeat value, i.e., there are $i < j$ such that $x_i = x_j$. Once this happens, the sequence cycles from x_i to x_{j-1} .

Find μ (the start of cycle) and λ (the cycle length) given $f(x)$ and x_0 .

Ex.:

$f(x) = (7x + 5) \% 12$ and $x_0 = 4$.

$f(x) = (26x + 11) \% 80$ and $x_0 = 7$.



Floyd's Cycle-Finding Algorithm

Two pointers: tortoise and hare (rabbit)

Uma descrição pode ser vista [aqui](#) (seção 16.4)